

## Описание задачи:

Разработать веб-приложение для преподавателя (ментора) и студентов. Приложение должно заменить устаревшую систему, предоставив современный, интуитивно понятный интерфейс. Основной функционал: управление дисциплинами, группами и студентами, загрузка и структурирование учебных материалов (лекции, файлы, видео), создание гибких тестов (квизов) с весовыми коэффициентами, ограничениями по времени и попыткам, а также система отчетности и обратной связи.

## 1. Технологический стек и архитектура

- **Frontend:** JavaScript (React). Использовать современные подходы (функциональные компоненты, хуки). Можно использовать библиотеку компонентов (например, Ant Design, Material-UI или Tailwind CSS) для ускорения разработки и обеспечения современного UI.
- **Backend:** Node.js с использованием фреймворка Express.js.
- **Database:**
  - *Вариант 1 (Локальный):* SQLite (embedded) - идеально для вашего MacBook и текущего количества студентов (10-15). Не требует отдельного сервера и легко переносится. Данные хранятся в файле `.db`.
  - *Вариант 2 (Удаленный/Timeweb):* PostgreSQL или MySQL. Timeweb, как и любой хостинг, поддерживает базы данных MySQL/PostgreSQL. Это более надежный и масштабируемый вариант для будущего.
- **Рекомендуемая архитектура:**
  - **Структура БД (основные сущности):**
    - `users` (id, login, password\_hash, role: 'mentor' | 'student', full\_name, email, group\_id)
    - `groups` (id, name)

- `disciplines` (id, name)
- `lectures` (id, discipline\_id, title, description, order\_index)
- `materials` (id, lecture\_id, file\_name, file\_path, file\_type)
- `tests` (id, lecture\_id, time\_limit\_minutes, session\_lifetime\_minutes, start\_datetime, attempts\_allowed, is\_active)
- `questions` (id, test\_id, question\_text, weight)
- `answers` (id, question\_id, answer\_text, is\_correct)
- `user_test_attempts` (id, user\_id, test\_id, attempt\_number, score, is\_completed)
- `user_answers` (id, attempt\_id, question\_id, selected\_answer\_id, is\_correct)
- **Структура папок проекта:**
  - `/backend` - серверная часть.
  - `/frontend` - клиентская часть.
  - `/uploads` - папка для хранения загруженных файлов (лекции, видео).
- **API:** RESTful API с четкими эндпоинтами (например, `/api/students`, `/api/tests`, `/api/reports`).

## 2. Основные разделы и роли пользователей

Система имеет два типа учетных записей:

### 1. Ментор (Преподаватель):

- Вход через логин/пароль, созданные при первом запуске системы.
- **Стартовая страница (Dashboard):** Краткая сводка по активности студентов, последние результаты, быстрый доступ к ключевым функциям.
- Функционал: Управление всеми разделами системы (см. пункт 3).

### 2. Студент:

- Вход через логин/пароль, выданные ментором.

- **Личный кабинет:** Ограниченный функционал, доступный только для изучения материалов и прохождения тестов.

### 3. Личный кабинет Ментора (Полный функционал)

Ментор имеет доступ ко всем разделам через главное навигационное меню.

#### 3.1. Управление Дисциплинами

- Просмотр списка существующих дисциплин.
- **Добавление дисциплины:** При добавлении не привязываемся к учебному году. Просто создаем название (например, "Высшая математика").

#### 3.2. Управление Группами и Студентами

- **Добавление группы:** Создание новой группы (например, "Группа Б-21").
- **Просмотр состава группы:** Клик по группе открывает список студентов.
- **Добавление студента в группу:**
  - Форма с обязательными полями: **Логин** (уникальный), **Пароль** (генерируется автоматически или вводится вручную), **ФИО**.
  - Необязательное поле: **Подробная информация** (поле textarea для любых заметок).
  - При создании студенту автоматически присваивается роль 'student'.
- **Редактирование/Удаление** студентов и групп.

#### 3.3. Управление Лекциями и Материалами (В контексте дисциплины)

- Просмотр дисциплины открывает список лекций.
- **Добавление лекции:**
  - Поле "Заголовков" (например, "Лекция 1. Введение").

- Возможность **загружать файлы**: PDF, PowerPoint, Word, Excel, а также видео-файлы. Обеспечить поддержку разных форматов через мульти-загрузку.
- Загруженные файлы должны отображаться под заголовком лекции с указанием типа файла.
- **Порядок лекций**: Автоматически сортировать по дате добавления или предоставить возможность ручной сортировки.

### 3.4. Создание и управление Тестами (Квизами)

Тест привязывается к конкретной **лекции** (под каждой лекцией может быть только один тест, но с возможностью его редактирования и пересоздания).

- **Создание теста**:
  - Поле: **Количество попыток** (целое число).
  - Поле: **Время жизни сессии** (в минутах, например, 10). Если студент не завершил тест за это время, сессия закрывается, и фиксируются данные на момент закрытия.
  - Поле: **Дата и время доступности** (старт тестирования). Формат: YYYY-MM-DD HH:MM.
- **Добавление вопросов в тест**:
  - Каждый вопрос содержит: Текст вопроса, **Вес** (число баллов за верный ответ).
  - Варианты ответов: Неограниченное количество. Один вариант помечается ментором как **верный** (галочка/radio button). Для студента все варианты выглядят одинаково.
- **Проверка теста**: Результат студенту показывается мгновенно ("Верно" / "Не верно") после нажатия на "Проверить" (вы можете реализовать проверку по одному вопросу или по всему тесту сразу).
- **Расчет итоговой оценки**:
  - На основе суммы баллов (с учетом весов) за правильные ответы.

- Конвертация в 5-балльную систему (например, 0-20% = 1, 20-40% = 2, 40-60% = 3, 60-80% = 4, 80-100% = 5). Алгоритм должен быть гибким и настраиваемым в будущем.

### 3.5. Функция "Пересдача"

- После завершения времени жизни теста у ментора должна быть возможность **назначить пересдачу**.
- Можно выбрать:
  - "Для всех студентов".
  - "Для выбранных студентов" (чтобы дать шанс только тем, кто не сдал или хочет улучшить результат).
- Для пересдачи заново задаются: **Новая дата и время старта, Время жизни сессии и Количество попыток**.

### 3.6. Отчеты

- Вкладка "Отчеты" в личном кабинете ментора.
- **Основные отчеты:**
  - Успеваемость по группе (список студентов с итоговыми оценками по каждой дисциплине).
  - Успеваемость по дисциплине (список студентов и их прогресс по каждой лекции/тесту).
  - Выборки: "Не сдавшие тест X", "Студенты с наилучшими результатами".
- **Экспорт:** Возможность экспортировать отчеты в форматы CSV или PDF.

### 3.7. Обратная связь

- Вкладка или кнопка "Обратная связь".
- Интерфейс для студента: Отправить сообщение ментору.
- Интерфейс для ментора: Получение списка сообщений от студентов. Отправка ответов. Реализовать через почтовый клиент (SMTP) или сохранять сообщения в базе данных, а ментору приходить уведомление на email.

## 4. Личный кабинет Студента

- **Стартовая страница:** Список **Дисциплин**, к которым он привязан (через группу).

- **Просмотр Дисциплины:** Разворачивается в список **Лекций**.
- **Материалы лекции:** Под названием лекции отображаются все загруженные ментором файлы (PDF, видео и т.д.). Они должны открываться для просмотра или скачивания.
- **Тестирование:**
  - Под каждой лекцией отображается ссылка на тест.
  - **Активность ссылки:** Ссылка активна **только** в промежутке [Дата и время старта] и [Дата и время старта + Время жизни сессии]. **Все остальное время ссылка неактивна (заблокирована).**
  - Даже если ссылка неактивна, под ней **всегда отображается** расшифровка: ответы студента на все вопросы, количество баллов по каждому ответу и итоговая оценка (если тест уже был пройден).
- **Сохранение:** В случае прерывания теста (закрытие браузера) прогресс должен сохраняться.

## 5. Дополнительные требования и логика

- **Безопасность:** Пароли должны храниться в хэшированном виде (bcrypt).
- **Валидация:** На всех этапах, как на клиенте, так и на сервере.
- **Адаптивность:** Приложение должно корректно отображаться на экранах разных размеров (ноутбук, монитор).
- **Обработка ошибок:** Выводить понятные сообщения об ошибках (например, "Неверный логин или пароль").

## 6. Что взять из "старого" приложения (скриншоты)

1. **Идея навигации:** Крупные, понятные иконки/названия разделов (Тестирование, Редактирование, Журнал и т.д.)

- использовать их как основу для главного меню, но с **современным стилем**.

2. **Структура "Журнала"**: Идея отображения результатов студентов в виде матрицы ("ячейка" - результат студента по конкретному тесту) - реализовать с помощью интерактивной таблицы, где по клику на ячейку можно увидеть детальный ответ.

## 7. Инструкция для запуска (для разработчика)

1. Выполнить `npm install` в папках `/backend` и `/frontend`.
2. Настроить переменные окружения (файл `.env`) для порта бэкенда и подключения к БД.
3. Запустить сервер (`node server.js` или `npm start`).
4. Запустить клиент (`npm start`).
5. При первом запуске автоматически создать учетную запись ментора (логин: `admin`, пароль: `admin` или предложить создать через форму регистрации).